

Compiling The Transverse-Field Ising Model

William Schjerve Moe, Yang Liu

(Dated: June 16, 2026)

Quantum simulation is a central application of quantum computing, as quantum systems are naturally represented on qubit registers but are often costly to simulate classically. To implement a simulated time evolution on gate-based quantum hardware, the target unitary must first be decomposed into gates from the available gate set. In this report we implement and compare three synthesis methods for the time evolution of the one-dimensional transverse-field Ising model (TFIM) with nearest-neighbour coupling: a naive method, a structure-preserving decomposition (SPD), and first-order Trotterisation. The naive method is exact up to numerical precision but scales exponentially, $\mathcal{O}(4^n)$, in both gate count and decomposition time, limiting the implementation to $n = 12$ qubits in our experiments. The structure-preserving method is likewise exact up to numerical precision, but exploits the fact that the TFIM dynamical Lie algebra is isomorphic to $\mathfrak{so}(2n)$. By performing the decomposition in $SO(2n)$, the method avoids the full 2^n Hilbert-space representation and achieves polynomial decomposition-time $\mathcal{O}(n^2)$, evaluated up to $n = 193$. The total runtime is dominated by the construction of the dynamical Lie algebra (DLA), which accounts for $\sim 99\%$ of the full pipeline time. Trotterisation has negligible decomposition time, but is approximate, with a reconstruction error scaling as $1/r$. The results show that exploiting Lie-algebraic structure can turn an otherwise exponential exact synthesis problem into a polynomial structure-preserving decomposition, while Trotterisation remains a useful approximate alternative when a tradeoff between accuracy and circuit cost is acceptable.

I. INTRODUCTION

Quantum simulations are one of the main motivations for quantum computing. In general, the classical resources required to simulate an arbitrary n -qubit system grow exponentially with n , whereas quantum computers naturally represent such systems using qubits. However, the desired unitary $U(t)$ must typically be decomposed into a sequence of elementary gates, and the number of required gates may grow rapidly with the system size n , the evolution time, and the desired accuracy. This gives rise to a compilation problem.

The Hamiltonian H determines how a quantum state evolves in time through the unitary time-evolution operator

$$U(t) = e^{-iHt}.$$

To simulate a quantum system on a quantum computer, the aim is therefore to approximate and implement this operator.

In this report, we study the compilation of the one-dimensional transverse-field Ising model (TFIM). The corresponding time-evolution operator cannot generally be implemented directly on gate-based hardware, and must instead be decomposed into a sequence of elementary gates. We compare three different approaches for compiling the TFIM time evolution. The first is a naive unitary synthesis method, which treats $U(t)$ as a generic element of $U(2^n)$. The second method is first-order Trotterisation, which approximates the time evolution. Finally, the third method is a structure-preserving decomposition (SPD), which exploits the special Lie-algebraic structure of the TFIM. Our goal is to compare these methods in terms of reconstruction error, gate count and decomposition time, allowing us to study the tradeoff be-

tween them.

The report is structured as follows. Section II (Theory) presents the theoretical background, including unitary synthesis, the transverse-field Ising model, Trotterisation, Lie algebras, and Cartan decompositions. Section III (Methods) describes the three compilation methods considered in this report: naive unitary synthesis, first-order Trotterisation, and the structure-preserving decomposition. Section IV (Results) presents the numerical results, including the small-system validation, Trotter convergence, and the comparison of the three methods in terms of reconstruction error, gate count, and decomposition time. Finally, Section V (Conclusion) summarises the main findings and discusses possible directions for further work.

II. THEORY

This section introduces the theoretical background needed for the three synthesis methods studied in this report. First, the time evolution of a quantum system and unitary synthesis is described, before introducing the transverse-field Ising model. The first-order Trotterisation is then introduced as the standard approach to Hamiltonian simulation, and finally Lie algebras and Cartan decompositions are explored as the mathematical framework for both the naive and structure-preserving recursive synthesis methods.

A. Unitary synthesis

For a Hamiltonian H of a quantum system, the time evolution of a state $|\psi(t)\rangle$ is governed by the Schrödinger

equation

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = H |\psi(t)\rangle,$$

The general solution of this equation can be written as

$$|\psi(t)\rangle = U(t) |\psi_0\rangle,$$

where $U(t)$ is a unitary operator describing the time evolution of the Hamiltonian and $|\psi_0\rangle$ is the initial quantum state. If H is time-independent, the time-evolution operator is given by

$$U(t) = e^{-iHt/\hbar}. \quad (1)$$

In the rest of the report, we work in units where $\hbar = 1$, so $U(t)$ becomes $U(t) = e^{-iHt}$.

Unitary synthesis is the process of decomposing an arbitrary target unitary into a sequence of quantum gates that can be implemented as a quantum circuit on a given quantum hardware platform. This task arises due to the fact that quantum hardware cannot directly implement arbitrary unitary matrices, but is restricted to a limited collection of operations, or quantum logic gates. For a quantum computer to run a quantum algorithm, the algorithm must therefore be expressed as a circuit composed of such elementary gates. Since different quantum hardware support different gate sets, the synthesis of a given unitary is also hardware-dependent.

A gate set is called universal if any n -qubit unitary can be approximated arbitrarily well by circuits using only gates contained in the set [1]. Given a universal gate set S and a target unitary $U \in U(2^n)$, unitary synthesis aims to find a circuit, C , consisting of gates from S :

$$C = (s_1, \dots, s_m), \quad s_i \in S.$$

The unitary implemented by this circuit is

$$U_C = s_m \cdots s_1$$

where the goal is to choose gates in C such that

$$d(U, U_C) \leq \epsilon,$$

for an accuracy ϵ . Here $d : U(2^n) \times U(2^n) \rightarrow \mathbb{R}$ is a metric on the unitary group and ϵ is a desired accuracy. If $\epsilon = 0$, the synthesis is exact; otherwise, it is known as an approximate synthesis [2].

B. The Transverse-Field Ising Model

The Ising model is a model in statistical physics used to describe ferromagnetism. In this report, we focus on the 1D transverse-field Ising model (TFIM), a quantum generalisation of the 1D Ising model for a linear chain of particles with magnetic spin. The model only considers neighboring interactions through a coupling parameter

J . In addition, an external magnetic field is applied perpendicular to the axis of the interaction. For a chain of n spin- $\frac{1}{2}$ magnetic moments, the Hamiltonian is given by

$$H = -J \sum_{j=0}^{n-2} Z_j Z_{j+1} - h \sum_{j=0}^{n-1} X_j, \quad (2)$$

where X, Z are Pauli operators, J is the coupling strength and h is the parameter of the magnetic field.

Since the TFIM Hamiltonian is time-independent, its time evolution is given by eq. (1).

C. Trotterisation

When implementing the time evolution e^{-iHt} of a quantum system on gate-based quantum hardware, one of the standard approaches is Trotterisation, also known as the product formula method. The approach is based on the Trotter product formula [3], while in the context of quantum simulation, Lloyd showed in 1996 that local Hamiltonians can be simulated efficiently on a quantum computer by decomposing the time evolution into a sequence of simple, implementable operators [4]. Trotterisation exploits the additive structure of the Hamiltonian by expressing it as a sum $H = \sum_k H_k$ where the individual exponents $e^{-iH_k t}$ can typically be implemented more efficiently than the full evolution. This method has become a fundamental tool in quantum simulation, and remains a standard approach for hardware implementations, due to its low circuit depth and natural compatibility with local Hamiltonians [5].

Suppose we have two operators A and B that commute, $[A, B] = 0$. Then the exponential of their sum factorises exactly:

$$e^{-i(A+B)t} = e^{-iAt} e^{-iBt}.$$

If the operators do not commute, $[A, B] \neq 0$, this factorisation no longer holds exactly [?]. The Baker-Campbell-Hausdorff formula [6] gives the correction:

$$e^A e^B = e^{A+B+\frac{1}{2}[A,B]+\dots}.$$

Thus, the leading error is governed by the commutator $[A, B]$, while the higher-order corrections involve nested commutators.

The first-order Trotter formula approximates the time evolution by dividing the total time t into r small steps of size $\delta t = t/r$ such that [7]

$$e^{-i(A+B)t} \approx (e^{-iA\delta t} e^{-iB\delta t})^r. \quad (3)$$

The approximation error over r steps scales as $\mathcal{O}(t^2/r)$, and so increasing r improves accuracy at the cost of a deeper circuit. Approximating $e^{-iHt} \approx (\prod_k e^{-iH_k t/r})^r$, incurs an error

$$\left\| e^{-iHt} - \left(\prod_k e^{-iH_k t/r} \right)^r \right\| = \mathcal{O}\left(\frac{t^2}{r} \sum_{i < j} \| [H_i, H_j] \| \right),$$

i.e. linear in $1/r$ at fixed t [4].

Higher-order Trotterisation formulas can reduce this error further, but we will only consider first-order Trotterisation for simplicity.

D. Cartan Decompositions for Unitary Synthesis

A widely used mathematical structure for synthesis methods is the KAK decomposition, also known as the Cartan decomposition. These decompositions come from Lie group theory and provide a structured way to factorize unitary operators into simpler components. This section will first introduce the necessary Lie algebra concepts and then present the generic Cartan decomposition method.

1. Lie algebra

A Lie algebra $\mathcal{L}$ is a vector space over a field $\mathbb{K}$ equipped with a bilinear product, $[\cdot, \cdot] : \mathcal{L} \times \mathcal{L} \rightarrow \mathcal{L}$ which satisfies antisymmetry and the Jacobi identity:

1. $[x, y] = -[y, x] \quad \forall x, y \in \mathcal{L}$,
2. $[x, [y, z]] + [y, [z, x]] + [z, [x, y]] = 0 \quad \forall x, y, z \in \mathcal{L}$.

The bilinear product is called the Lie bracket.

One of the central examples of the Lie bracket is the matrix commutator. If A and B are square matrices, their commutator is defined as

$$[A, B] = AB - BA.$$

Since the matrix commutator satisfies the Lie algebra axioms, a vector space of matrices closed under the commutator forms a Lie algebra when equipped with the commutator. Moreover, if $\mathfrak{g}$ is a Lie algebra, a subalgebra $\mathfrak{k} \subseteq \mathfrak{g}$ is a subspace that is closed under the bilinear product. This means that if $X, Y \in \mathfrak{k}$ then $[X, Y] \in \mathfrak{k}$.

For unitary synthesis, we will consider matrix Lie groups, which are closed subgroups of all invertible matrices

$$GL(N, \mathbb{C}) = \{A \in \mathbb{C}^{N \times N} \mid \det(A) \neq 0\}.$$

The main example of a matrix Lie group is the unitary group

$$U(N) = \{U \in GL(N, \mathbb{C}) \mid U^\dagger U = I\}.$$

This is the group of all $N \times N$ unitary matrices and is important for quantum computing as n -qubit gates are represented by unitary matrices in $U(2^n)$.

For a matrix Lie group $\mathcal{G}$, there is an associated Lie algebra $\mathfrak{g}$ given by

$$\mathfrak{g} = \{X \in \mathbb{C}^{N \times N} \mid e^{tX} \in \mathcal{G} \text{ for all } t \in \mathbb{R}\}.$$

The map from a Lie algebra to its Lie group is the exponential map

$$\exp : \mathfrak{g} \rightarrow \mathcal{G}, \quad X \mapsto e^X,$$

which, for matrix Lie groups, is the matrix exponential.

For the unitary group, the corresponding Lie algebra consists of skew-Hermitian matrices

$$\mathfrak{u}(N) = \{X \in \mathbb{C}^{N \times N} \mid X^\dagger = -X\}.$$

If $X \in \mathfrak{u}(N)$, then

$$(e^X)^\dagger e^X = e^{X^\dagger} e^X = e^{-X} e^X = I.$$

Thus $e^X \in U(N)$.

This matches the form of the time evolution of the Hamiltonian in eq. (1). Since Hamiltonians are Hermitian, $-iH$ is skew-Hermitian. Thus $-iH \in \mathfrak{u}(N)$, while the corresponding time-evolution operator $U(t) \in U(N)$. This connection is one reason Lie algebras are useful in quantum circuit synthesis [8].

For Hamiltonian simulation, we are usually interested in unitary matrices up to a global phase. It is therefore useful to introduce the special unitary group

$$SU(N) = \{U \in U(N) \mid \det(U) = 1\},$$

whose Lie algebra is

$$\mathfrak{su}(N) = \{X \in \mathbb{C}^{N \times N} \mid X^\dagger = -X, \text{tr}(X) = 0\}.$$

This is the algebra of traceless skew-Hermitian matrices and has dimension

$$\dim(\mathfrak{su}(N)) = N^2 - 1. \quad (4)$$

We will also use the special orthogonal group

$$SO(N) = \{O \in \mathbb{R}^{N \times N} \mid O^T O = I, \det(O) = 1\}.$$

Its Lie algebra is

$$\mathfrak{so}(N) = \{X \in \mathbb{R}^{N \times N} \mid X^T = -X\},$$

which consists of real skew-symmetric matrices and has the dimension

$$\dim(\mathfrak{so}(N)) = \frac{N(N-1)}{2}. \quad (5)$$

The elements of the Lie algebra are often called generators, because exponentiating them gives elements of the corresponding Lie group. In quantum computing, gates can be written as exponentials of such generators. For example, the single-qubit rotation gates are

$$R_x(\theta) = e^{-i\theta X/2}, \quad R_y(\theta) = e^{-i\theta Y/2}, \quad R_z(\theta) = e^{-i\theta Z/2}.$$

Here $X/2$, $Y/2$ and $Z/2$ are the Hermitian generators for the rotations, while $-iX/2$, $-iY/2$ and $-iZ/2$ are the corresponding skew-Hermitian generators in $\mathfrak{u}(2)$. Thus, we can think of unitary synthesis as expressing a complicated unitary exponential using a product of simpler exponentials.

2. The KAK decomposition

Let $\mathcal{G}$ be a Lie group with Lie algebra $\mathfrak{g}$, and let $\mathcal{K} \subset \mathcal{G}$ be a subgroup with Lie subalgebra $\mathfrak{k}$. If we equip $\mathfrak{g}$ with an appropriate inner product (often the Hilbert-Schmidt inner product), $\mathfrak{g}$ can be written using an orthogonal decomposition as

$$\mathfrak{g} = \mathfrak{k} \oplus \mathfrak{p} \quad (6)$$

where $\mathfrak{p}$ is the orthogonal complement of $\mathfrak{k}$ with respect to the inner product:

$$\mathfrak{p} = \{X \in \mathfrak{g} \mid \langle X, Y \rangle = 0 \ \forall Y \in \mathfrak{k}\}.$$

If $\mathfrak{p}$ and $\mathfrak{k}$ satisfy the commutation relations

$$\begin{aligned} [\mathfrak{k}, \mathfrak{k}] &\subset \mathfrak{k}, \\ [\mathfrak{k}, \mathfrak{p}] &\subset \mathfrak{p}, \\ [\mathfrak{p}, \mathfrak{p}] &\subset \mathfrak{k}, \end{aligned}$$

the decomposition eq. (6) is called Cartan decomposition.

Given a Cartan decomposition, let $\mathfrak{a} \subseteq \mathfrak{p}$, be a maximal abelian subspace. This means all elements of $\mathfrak{a}$ commute with each other

$$[A, B] = 0 \quad \forall A, B \in \mathfrak{a}$$

and that $\mathfrak{a}$ cannot be enlarged within $\mathfrak{p}$ while preserving this property. Exponentiating this subalgebra gives the abelian subgroup,

$$\mathcal{A} = \exp(\mathfrak{a}).$$

Given such a Cartan decomposition and a maximal abelian subspace $\mathfrak{a} \subseteq \mathfrak{p}$, the KAK decomposition states that any group element $G \in \mathcal{G}$ can be decomposed as

$$G = K_1 A K_2, \quad (7)$$

where $K_1, K_2 \in \mathcal{K}$ and $A \in \mathcal{A}$ [9].

This factorization is often used for unitary synthesis as the middle factor A is generated by commuting elements, while the outer factors K_1 and K_2 lie in the subgroup $\mathcal{K}$. In recursive Cartan decomposition methods, these outer factors are recursively decomposed until the remaining factors correspond to implementable quantum gates.

3. KAK decompositions used in this work

The KAK decomposition does not correspond to a single universal factorization. Instead, the form of the decomposition depends on the chosen Lie group $\mathcal{G}$ and subgroup $\mathcal{K}$. Wierichs et al. classify KAK decompositions of classical compact Lie groups into ten distinct types, depending on the choice of $\mathcal{G}$ and $\mathcal{K}$ [9]. Table I shows the three different decomposition types used in this report. Type A and type AIII are used together in an

TABLE I. The three KAK decomposition types used in this report, adapted from [9]. The type A and AIII decompositions are used for general unitary synthesis of n -qubit systems, while BDI is used for the structure-preserving synthesis of the TFIM time-evolution operator.

Type	$\mathcal{G}$	$\mathcal{K}$
A	$U(d) \times U(d)$	$U(d)$
AIII	$U(p+q)$	$U(p) \times U(q)$
BDI	$SO(p+q)$	$SO(p) \times SO(q)$

alternating recursive scheme for general n -qubit unitary synthesis. This method treats the TFIM time-evolution as an arbitrary element of $U(2^n)$. It is therefore broadly applicable, but scales poorly because it works in the full 2^n -dimensional Hilbert space. Type BDI is used in the structure-preserving TFIM method after mapping the TFIM dynamics to a $2n \times 2n$ orthogonal Majorana representation. In this representation, the recursive decomposition preserves the $\mathfrak{so}(2n)$ structure. We describe these two recursive schemes in section II E and section III C.

E. A general KAK decomposition

The central idea of recursive unitary synthesis is to apply KAK decompositions repeatedly to reduce an n -qubit unitary all the way down to implementable 2×2 single-qubit gates. In this work a general KAK decomposition of a n -qubit unitary uses an alternating sequence of type AIII and type A decompositions from table I.

The type AIII decomposition of a unitary $U \in U(2^n)$ yields [9]

$$U = K_1 A_{\text{AIII}} K_2, \quad K_1, K_2 \in U(2^{n-1}) \times U(2^{n-1}),$$

and a type A decomposition of a block diagonal element $K \in U(2^{n-1}) \times U(2^{n-1})$ yields

$$K = K'_1 A_A K'_2, \quad K'_1, K'_2 \in U(2^{n-1}).$$

The AIII step reduces a general n -qubit unitary into block-diagonal $(n-1)$ -qubit unitaries and a Cartan factor that mixes the two blocks. In the decomposition the block-diagonal structure of K_1 and K_2 means that the two $(n-1)$ -qubit unitaries act on separate subspaces, selected by the state of the first qubit. The remaining Cartan factor A_{AIII} , mixes the two subspaces. In circuit representation this factor is implemented as a uniformly controlled R_Y rotation on the first qubit, conditioned on the remaining $n-1$ qubits. A uniformly controlled rotation, also known as a Select operator, or multiplexer, consists of 2^{n-1} independent rotations, one for each computational basis state in the control register [10]. This is illustrated in fig. 1.

In the type A decomposition, the Cartan subgroup element $A_A \in \exp(\mathfrak{a}_A)$ takes the form of a uniformly controlled R_Z rotation on the first qubit. As a result, the type A decomposition isolates the action of the first qubit

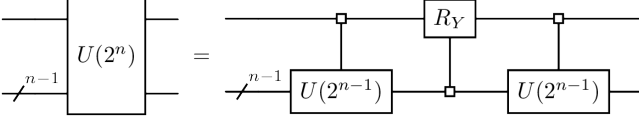


FIG. 1. A quantum circuit illustrating the AIII decomposition of a unitary matrix $U(2^n)$, where n is the number of qubits. This results in two controlled unitaries $U(2^{n-1})$ that act as the K s and a controlled R_Y acting as the A in KAK .

into a small set of commuting controlled gates, while the remaining operations act only on the $n - 1$ remaining qubits. Effectively, it takes a controlled unitary $U(2^{n-1})$, resulting from an AIII decomposition seen in fig. 1, and decouples it from the first qubit. This is illustrated in fig. 2.

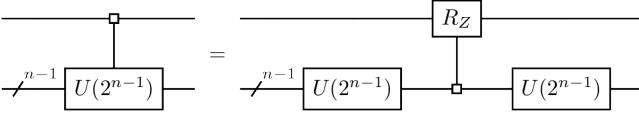


FIG. 2. A quantum circuit illustrating the A decomposition of a controlled unitary matrix $U(2^{n-1})$, where n is the number of qubits. The decomposition effectively "decouples" the first qubit from the $n - 1$ others, leaving two unitaries $U(2^{n-1})$ that act as the K s, and a controlled R_Z rotation acting as the A in KAK .

This interlocking structure enables a recursion between these two types of decompositions [9]:

$$U(2^n) \xrightarrow{\text{AIII}} U(2^{n-1}) \times U(2^{n-1}) \xrightarrow{A} U(2^{n-1}) \xrightarrow{\text{AIII}} \dots \\ \xrightarrow{\text{AIII}} U(2) \times U(2) \xrightarrow{A} U(2).$$

At each level, the abelian elements A are kept unchanged, while the two subgroup components K_1 and K_2 are recursively decomposed. Repeating this process reduces the qubit count at each level until only 2×2 unitaries remain.

This alternating AIII-A scheme is the mathematical backbone of several established synthesis algorithms, including the Quantum Shannon Decomposition [9] and the Block-ZXZ decomposition, which differ mainly in their choice of Cartan subalgebra at each level. For the TFIM specifically, the structure of the Hamiltonian allows for a different and more effective recursion, which we describe next.

III. METHODS

This section presents three different approaches for compiling the TFIM time-evolution operator in eq. (2). The first is a naive synthesis method, which treats the time evolution as a generic element of $U(2^n)$. The second

is first-order Trotterisation, which exploits the decomposition of the Hamiltonian into local terms but introduces an approximation error. The third is the structure-preserving decomposition, which uses the special Lie-algebraic structure of the TFIM to represent the dynamics in $SO(2n)$ before mapping the result back to a qubit circuit. All code is available on GitHub¹.

A. Naive unitary synthesis

Starting from the Ising Hamiltonian of eq. (2), we form the $2^n \times 2^n$ unitary $U(t) = e^{-iHt}$ of eq. (1) and decompose it with the alternating AIII / Type-A decomposition from section II E into 2×2 unitaries from the $K \in \mathcal{K}$ and uniformly controlled rotations, resulting from the $A \in \exp(\mathfrak{a})$ decomposition elements. The 2×2 unitaries are decomposed into single-qubit rotations by the ZYZ decomposition $U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$. Since our error metric is phase-aligned, the global phase α is ignored. The uniformly controlled rotations are implemented by Gray-code recursion, interleaving single-qubit rotations with CNOTs. The circuit is thus expressed in the native gate set $\{R_z, R_y, \text{CNOT}\}$. For $n \leq 7$ we verify the result by reconstructing $U(t)$ from the gate list in PennyLane and comparing against the exact operator via the phase-aligned error of eq. (16).

B. First-order Trotterisation

For the transverse-field Ising model with the Hamiltonian in eq. (2), the two natural terms are $H_{ZZ} = -J \sum_i Z_i Z_{i+1}$ and $H_X = -h \sum_i X_i$. These terms do not commute in general, as $[Z_i Z_{i+1}, X_i] \neq 0$. Therefore, the full time-evolution operator cannot be factorised exactly as $e^{-iHt} = e^{-iH_{ZZ}t} e^{-iH_X t}$. Applying the first-order Trotter formula as in eq. (3) we approximate

$$e^{-iHt} \approx (e^{-i\delta t H_{ZZ}} e^{-i\delta t H_X})^r,$$

with $\delta t = t/r$ and r being the number of steps. The key practical advantage is that each factor separates into commuting Pauli terms. All $Z_i Z_{i+1}$ terms commute with one another, and similarly all X_i commute with one another. Each term therefore exponentiates to a known gate directly:

$$e^{-i\delta t (-J) Z_i Z_{i+1}} = R_{ZZ}(2J\delta t), \\ e^{-i\delta t (-h) X_i} = R_X(2h\delta t),$$

where R_{ZZ} decomposes into CNOT - R_Z - CNOT, and R_X is a single qubit gate. One Trotter step therefore

¹ https://github.com/willism/Compiling_The_Transverse-Field_Ising_Model

consists of n single-qubit R_X gates, followed by $(n-1)$ two-qubit R_{ZZ} gates. This gives a gate count of

$$n_{\text{rot}} = r \cdot (2n - 1),$$

scaling linearly in both n and r .

C. Structure-preserving decomposition (SPD)

The structure-preserving decomposition exploits the special Lie-algebraic structure of the transverse-field Ising model. Instead of treating the time-evolution operator as a generic element of $U(2^n)$, the TFIM time-evolution is represented in the orthogonal group $SO(2n)$. Then a recursive BDI Cartan decomposition is performed in this orthogonal representation, before the resulting rotations are mapped back to Pauli rotations on qubits. The method thus preserves the algebraic structure of the TFIM throughout the decomposition. This section is inspired by the structure-preserving approach developed by Wierichs et al. [9].

1. Lie-algebraic structure of the TFIM

Given a n -qubit Hamiltonian H acting on the Hilbert space $(\mathbb{C}^2)^{\otimes n}$. Since Pauli strings form a basis for n -qubit operators, any spin Hamiltonian can be written as a linear combination of Pauli strings [11], $P_k \in \{I, X, Y, Z\}^{\otimes n}$,

$$H = \sum_{k=1}^K c_k P_k, \quad c_k \in \mathbb{R}.$$

The set $\mathcal{P} = \{P_k\}_{k=1}^K$ denotes the Pauli strings appearing in the Hamiltonian, and the dynamical Lie algebra (DLA) associated with $\mathcal{P}$ is then defined as the smallest Lie subalgebra of $\mathfrak{su}(2^n)$ containing iP_k

$$\mathfrak{g} = i\langle \mathcal{G} \rangle_{\text{Lie}} = \langle iH_1, \dots, iH_K \rangle_{\text{Lie}} \subseteq \mathfrak{su}(2^n).$$

Here $\langle \cdot \rangle_{\text{Lie}}$ denotes closure under real linear combinations and commutators. This means that the DLA contains the skew-Hermitian generators, all commutators between them, all commutators of the commutators, and so on. Thus, the DLA describes the set of all skew-Hermitian operators that can be generated from the Hamiltonian terms through linear combinations and repeated commutators.

The Lie group associated with $\mathfrak{g}$ is a subgroup of $SU(2^n)$ and generated by exponentiating elements of $\mathfrak{g}$

$$\mathcal{G} = \{e^{iA} : iA \in \mathfrak{g}\} \subseteq SU(2^n)$$

This group contains the unitary transformations generated by the Hamiltonian terms. If the DLA is the full $\mathfrak{su}(2^n)$ algebra, then the generated unitary group is

$SU(2^n)$ and produces the dynamics of a full n -qubit unitary. In that case, from eq. (4), the dimension become

$$\dim(\mathfrak{su}(2^n)) = 4^n - 1,$$

which grows exponentially with the number of qubits. Therefore, when synthesising a generic Hamiltonian, one must work in an exponentially large unitary space.

On the other hand, if the DLA is a proper subalgebra of $\mathfrak{su}(2^n)$, the evolution of the Hamiltonian is restricted by the structure through symmetries or other constraints. Thus, the characterisation of the DLA can be crucial for unitary synthesis; instead of decomposing the time-evolution of the Hamiltonian as a generic $2^n \times 2^n$ unitary, one can instead work inside a smaller Lie group generated by the DLA.

The TFIM is an example of such a system. Its dynamical Lie algebra is not the full $\mathfrak{su}(2^n)$. Instead, it has been shown that, for open boundary conditions, it is isomorphic to the special orthogonal algebra $\mathfrak{so}(2n)$ [12]. Its Hamiltonian can be mapped, using the Jordan-Wigner transformation, to a free-fermion Hamiltonian. Under this mapping, each Pauli generator becomes a quadratic Majorana operator. Since the quadratic Majorana operators are closed under commutators, these terms form the free-fermion algebra $\mathfrak{g}^{FF}$ [13]. Since $\mathfrak{g}^{FF} \cong \mathfrak{so}(2n)$, we get

$$\mathfrak{g}_{\text{TFIM}} \cong \mathfrak{so}(2n)$$

From eq. (5) the dimension of $\mathfrak{so}(2n)$ is

$$\dim(\mathfrak{so}(2n)) = \frac{2n(2n-1)}{2} = n(2n-1).$$

This is the motivation for the structure-preserving method. Instead of treating the TFIM time evolution as a generic unitary in $U(2^n)$, the time-evolution is instead represented in a smaller space, whose dimension only grows quadratically with n .

For implementing the SPD, we use the rotated TFIM Hamiltonian, where a Hadamard gate is applied to every qubit. Then the Z -gates are mapped to X -gates and X -gates are mapped to Z -gates.

$$HZH = X, \quad HXH = Z$$

From the normal TFIM Hamiltonian eq. (2) we then obtain the rotated Hamiltonian

$$H_{\text{rot}} = -J \sum_{j=0}^{n-2} X_j X_{j+1} - h \sum_{j=0}^{n-1} Z_j. \quad (8)$$

Their time-evolution operators are related by

$$U_{\text{rot}}(t) = W U_{\text{TFIM}}(t) W^\dagger.$$

where $W = H^{\otimes n}$ is the global Hadamard transformation. Since W is a unitary, the original and rotated Hamiltonians are unitarily equivalent and describe the same physics

[14, p. 36]. The reason for using the new basis is that the terms Z_j and $X_j X_{j+1}$ have a direct mapping to quadratic Majorana operators.

The generating set of the rotated TFIM is

$$\mathcal{G}_{\text{rot}} = \{X_j X_{j+1}\}_{j=0}^{n-2} \cup \{Z_j\}_{j=0}^{n-1}.$$

Then the corresponding DLA is

$$\mathfrak{g} = \langle \{iX_j X_{j+1}\}_{j=0}^{n-2} \cup \{iZ_j\}_{j=0}^{n-1} \rangle_{\text{Lie}}.$$

To make this construction explicit, consider the $n = 2$ case. Here the rotated TFIM Hamiltonian is

$$H_{\text{rot}} = -JX_0X_1 - hZ_0 - hZ_1. \quad (9)$$

The skew-Hermitian generators then become

$$\{iX_0X_1, iZ_0, iZ_1\}$$

Taking commutators generates the additional Pauli strings:

$$[iZ_0, iX_0X_1] = -2iY_0X_1 \quad \text{and} \quad [iZ_1, iX_0X_1] = -2iX_0Y_1,$$

Further commutators gives

$$[iZ_0, iX_0Y_1] = -2iY_0Y_1.$$

This results in the DLA for the 2-qubit case:

$$\mathfrak{g}_{n=2} = \text{span}_{\mathbb{R}}\{iZ_0, iZ_1, iX_0X_1, iY_0X_1, iX_0Y_1, iY_0Y_1\} \quad (10)$$

These six generators are closed under the commutator and form an algebra isomorphic to $\mathfrak{so}(4)$. This matches the dimension formula $\dim(\mathfrak{so}(4)) = 6$ and illustrates how the DLA of the TFIM does not grow into the full $\mathfrak{su}(2^n)$ as for general unitaries. Instead, the DLA is inside the free fermionic algebra, which is isomorphic to $\mathfrak{so}(2n)$.

2. Majorana representation and mapping

Using the Jordan-Wigner transformation, the rotated Hamiltonian eq. (8) can be written using Majorana operators. For a system with n fermionic modes, there are $2n$ Majorana operators $\{c_j\}_{j \in \{1, 2n\}}$. These operators can be defined as combinations of the fermionic annihilation and creation operators f_i and $f_i^\dagger$,

$$c_{2j-1} = f_j + f_j^\dagger \quad c_{2j} = -i(f_j - f_j^\dagger),$$

for $j = 1, \dots, n$ [15]. The Majorana operators are Hermitian and satisfy the anticommutation relation

$$\{c_j, c_i\} = 2\delta_{i,j}$$

The Jordan-Wigner transformation gives an isomorphism between a system of N fermionic modes and N

spins [16]. For the Majorana operators, this transformation can be chosen in terms of Pauli operators as

$$c_{2j-1} = \left(\prod_{k=0}^{j-2} Z_k\right) Y_{j-1} \quad (11)$$

$$c_{2j} = \left(\prod_{k=0}^{j-2} Z_k\right) X_{j-1} \quad (12)$$

[17]. Using this convention, we find

$$c_{2j-1}c_{2j} = Y_{j-1}X_{j-1} = -iZ_{j-1}$$

$$c_{2j-1}c_{2(j+1)} = Y_{j-1}Z_{j-1}X_j = iX_{j-1}X_j$$

Equivalently, using zero-indexed qubit variable $r = j - 1$ both the Z_r terms and the nearest-neighbour terms $X_r X_{r+1}$ can be represented by quadratic Majorana operators.

$$Z_r = ic_{2r+1}c_{2r+2} \quad \text{and} \quad X_r X_{r+1} = -ic_{2r+1}c_{2r+4}$$

This allows us to rewrite the rotated TFIM Hamiltonian as

$$H_{\text{rot}} = iJ \sum_{r=0}^{n-2} c_{2r+1}c_{2r+4} - ih \sum_{r=0}^{n-1} c_{2r+1}c_{2r+2},$$

This shows that the TFIM Hamiltonian is quadratic in Majorana operators. These Hamiltonians are called free-fermion Hamiltonians. The corresponding DLA of the rewritten Hamiltonian is then generated by the quadratic Majorana terms appearing in $-iH_{\text{rot}}$. This gives

$$\mathfrak{g}_{\text{rot}} = \left\langle \{c_{2r+1}c_{2r+4}\}_{r=0}^{n-2}, \{c_{2r+1}c_{2r+2}\}_{r=0}^{n-1} \right\rangle_{\text{Lie}}.$$

To find the isomorphism to $\mathfrak{so}(2n)$ define the $2n \times 2n$ matrices F_{ij} as

$$F_{ij} = E_{ij} - E_{ji},$$

where E_{ij} are elementary matrices with 1 in entry (i, j) and zeros elsewhere. The matrices F_{ij} for $1 \leq i < j \leq 2n$ form a basis for $\mathfrak{so}(2n)$ [18, p. 92]. The isomorphism between the quadratic Majorana operators in $\mathfrak{g}$ and $\mathfrak{so}(2n)$ can then be defined as

$$\rho : \mathfrak{g} \rightarrow \mathfrak{so}(2n) : \rho(c_i c_j) = 2F_{i,j}. \quad (13)$$

This maps each quadratic Majorana operator to a plane rotation in $\mathfrak{so}(2n)$.

Continuing with the $n = 2$ case, the Majorana relations above give

$$Z_0 = ic_1c_2, \quad Z_1 = ic_3c_4$$

and

$$X_0X_1 = -ic_1c_4.$$

Therefore we can write the Hamiltonian as

$$H_{\text{rot}} = iJc_1c_4 - ihc_1c_2 - ihc_3c_4.$$

Using the isomorphism we obtain the element in $\mathfrak{so}(4)$

$$\rho(-iH_{\text{rot}}) = 2JF_{14} - 2hF_{12} - 2hF_{34}.$$

Represented as a 4×4 skew-symmetric matrix, this gives

$$\rho(-iH_{\text{rot}}) = \begin{pmatrix} 0 & -2h & 0 & 2J \\ 2h & 0 & 0 & 0 \\ 0 & 0 & 0 & -2h \\ -2J & 0 & 2h & 0 \end{pmatrix}. \quad (14)$$

Exponentiating this matrix gives the corresponding orthogonal time-evolution representation,

$$O(t) = \exp(t\rho(-iH_{\text{rot}})) \in SO(4).$$

3. Recursive BDI decomposition

After mapping the rotated TFIM Hamiltonian to the orthogonal representation, it can be represented as a real skew-symmetric matrix

$$\rho(-iH_{\text{rot}}) \in \mathfrak{so}(2n).$$

Exponentiating this generator gives an element of the special orthogonal group

$$O(t) = \exp(t\rho(-iH_{\text{rot}})) \in SO(2n).$$

Instead of using a generic unitary decomposition, the goal of the decomposition is to preserve the orthogonal matrix structure. Thus it is natural to use the BDI decomposition, which is a Cartan decomposition adapted to the orthogonal group. For a matrix $G \in SO(p+q)$ the BDI decomposition gives the factorization

$$G = K_1AK_2,$$

where

$$K_1, K_2 \in SO(p) \times SO(q)$$

while A belongs to the Cartan subgroup generated by mutually commuting rotations between the two subspaces of dimension p and q . Here the K -matrices preserve the block structure, while the A -factor contains the rotations coupling the two blocks [9].

In block form, the K_1 and K_2 are

$$K_1 = \begin{pmatrix} k_{11} & 0 \\ 0 & k_{12} \end{pmatrix}, \quad K_2 = \begin{pmatrix} k_{21} & 0 \\ 0 & k_{22} \end{pmatrix},$$

with

$$k_{11}, k_{21} \in SO(p), \quad k_{12}, k_{22} \in SO(q).$$

The Cartan factor A is a cosine-sine block. If $p = q$, it takes the simple balanced form

$$A = \begin{pmatrix} \cos(\Theta) & -\sin(\Theta) \\ \sin(\Theta) & \cos(\Theta) \end{pmatrix}, \quad (15)$$

where

$$\Theta = \text{diag}(\theta_1, \dots, \theta_p).$$

More generally, in the unbalanced case $p < q$, the Cartan factor can be written as

$$A = \begin{pmatrix} C & 0 & -S \\ 0 & I_{q-p} & 0 \\ S & 0 & C \end{pmatrix},$$

where

$$C = \text{diag}(\cos \theta_1, \dots, \cos \theta_p), \quad S = \text{diag}(\sin \theta_1, \dots, \sin \theta_p).$$

Thus the A -factor contains $p = \min(p, q)$ independent commuting plane rotations. This form is needed when a block has odd dimension.

Numerically, we implement the BDI decomposition using the cosine-sine decomposition. This decomposition returns block-orthogonal matrices $k_{11}, k_{12}, k_{21}, k_{22}$ and a set of angles θ_i which are put together using the K_1AK_2 form above. Since the cosine-sine decomposition may return blocks with determinant -1 , while the elements in $SO(p) \times SO(q)$ must have determinant $+1$, we normalize the determinant. This is done by flipping the signs of the first row or column of the k_{ij} -block. This sign change is then compensated by adjusting the Cartan angles. This moves the negative sign into the A -factor and ensures that the final K -blocks lie in the correct space.

After one BDI step, A is kept fixed, while the block matrices in K_1 and K_2 are smaller orthogonal matrices and can be decomposed further. This results in a recursive Cartan decomposition

$$SO(2n) \rightarrow SO(p) \times SO(q) \rightarrow SO(p_1) \times SO(q_1) \times \dots$$

The decomposition is continued until the blocks are one-(scalars) or two-dimensional. A two-dimensional special orthogonal matrix is a plane rotation

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix},$$

The result of the recursive BDI decomposition is thus a product of plane rotations in the $SO(2n)$ representation,

$$O(t) = \prod_{\ell} \exp(\theta_{\ell} F_{a_{\ell} b_{\ell}}),$$

where $F_{ab} = E_{ab} - E_{ba}$ generates a rotation in the (a, b) -plane. These rotations remain inside the $\mathfrak{so}(2n)$ representation of the TFIM dynamical Lie algebra, and preserve the structure of the original Hamiltonian.

For $n = 2$, there is only one BDI recursion

$$SO(4) \rightarrow SO(2) \times SO(2).$$

The decomposition then becomes

$$G = \begin{pmatrix} k_{11} & 0 \\ 0 & k_{12} \end{pmatrix} A \begin{pmatrix} k_{21} & 0 \\ 0 & k_{22} \end{pmatrix}.$$

Thus, the decomposition gives 4 rotations from the K -blocks. Moreover the A block as presented in eq. (15) contains two angles θ_1, θ_2 , which means it contains two plane rotations. Thus the result of the decomposition corresponds to 6 plane rotations. This is consistent with the size of the DLA.

4. Mapping back to Pauli rotations

After having obtained the BDI decomposition of the $2n \times 2n$ orthogonal time-evolution matrix as a product of plane rotations in $SO(2n)$, each rotation must be mapped back to the qubit representation. Each generator $F_{ab} \in \mathfrak{so}(2n)$ is isomorphic to a quadratic Majorana operator $c_a c_b$, and each quadratic Majorana operator corresponds to a Pauli string in the TFIM dynamical Lie algebra.

Using the construction of ρ in eq. (13), the inverse map, ρ^{-1} , gives

$$F_{ab} \mapsto \frac{1}{2} c_a c_b.$$

Each quadratic Majorana corresponds to a skew-Hermitian Pauli string

$$c_a c_b = -i s_{ab} P_{ab}, \quad s_{ab} \in \{\pm 1\}.$$

Combining this, each plane rotation maps back to a Pauli string as follows

$$\exp(\theta F_{ab}) \mapsto \exp\left(\frac{\theta}{2} c_a c_b\right) = \exp\left(-i \frac{s_{ab} \theta}{2} P_{ab}\right).$$

Here the angle θ corresponds to the angle from the BDI decomposition. Thus the corresponding Pauli rotation has the form

$$\exp(-i \alpha P_{ab}), \quad \alpha = \frac{s_{ab} \theta}{2}.$$

Thus, by mapping each rotation obtained through the BDI decomposition, the structure-preserving method gives the product

$$U_{\text{SPD}}(t) = \prod_{\ell} \exp(-i \alpha_{\ell} P_{\ell}),$$

where each P_{ℓ} is a Pauli string belonging to the TFIM dynamical Lie algebra. In this way, the decomposition preserves the algebraic structure of the original Hamiltonian from the $SO(2n)$ representation all the way back to the qubit circuit.

In our implementation, we first construct the full DLA generated by the rotated TFIM Hamiltonian terms. Then we manually build a dictionary between the Pauli strings and quadratic Majorana operators. This is done since the BDI decomposition output consists of plane rotations F_{ab} . To map these rotations to Pauli strings P_{ab} , we use this lookup map to make the mapping

$$F_{ab} \mapsto c_a c_b \mapsto P_{ab},$$

possible.

5. Full pipeline

To summarize, the full SPD pipeline is:

1. Rotate the TFIM Hamiltonian into the $XX + Z$ form.
2. Generate the TFIM dynamical Lie algebra.
3. Map the Pauli generators to quadratic Majorana operators.
4. Represent the resulting algebra inside $\mathfrak{so}(2n)$.
5. Exponentiate $\rho(-i H_{\text{rot}})$ to obtain $O(t) \in SO(2n)$.
6. Apply the recursive BDI decomposition to $O(t)$.
7. Map the resulting plane rotations back to Pauli rotations.

D. Metrics

When comparing the three methods, naive unitary synthesis (section III A), Trotterisation (section II C) and the structure-preserving decomposition (section III C), we need some metrics to compare them. We will consider the three metrics: reconstruction error, gate count, and decomposition time. For each of these metrics, we are especially interested in seeing how they *scale* with the number of qubits.

1. Reconstruction error

The reconstruction error quantifies how accurately a decomposition reproduces the target unitary. Let U_{ref} be the unitary we wish to implement, in our case the time-evolution operator $U_{\text{ref}} = e^{-iHt}$, and let U_{test} be the unitary actually realised by the decomposition. We measure their discrepancy with a distance d ,

$$\text{err} = d(U_{\text{ref}}, U_{\text{test}}).$$

We take d to be the distance induced by the Frobenius norm. For a matrix $A \in \mathbb{C}^{m \times n}$ with entries a_{ij} , the Frobenius norm is

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2},$$

and the associated distance between two matrices is $d(A, B) = \|A - B\|_F$.

Because our matrices are unitary, they are physically meaningful only up to a global phase: U and $e^{i\phi}U$ produce identical measurement statistics. We therefore remove this phase before measuring the error. Given U_{ref} and U_{test} , we determine the optimal global phase from their Hilbert-Schmidt overlap,

$$\phi = \arg \text{Tr} \left(U_{\text{test}} U_{\text{ref}}^\dagger \right),$$

and define the *phase-aligned* reconstruction error as

$$\text{err} = \|U_{\text{ref}} - e^{-i\phi} U_{\text{test}}\|_F. \quad (16)$$

If U_{test} differs from the target by only a global phase, i.e. $U_{\text{test}} = e^{i\phi} U_{\text{ref}}$, then $e^{-i\phi} U_{\text{test}} = U_{\text{ref}}$ and the error vanishes, as desired. The same expression is used for all three methods.

The test unitary U_{test} is obtained by multiplying together the primitive operations obtained by each decomposition in circuit order. Each operator is then expressed as an operator on the full 2^n dimensional Hilbert space. For the structure-preserving and Trotter decompositions this is a product of exponentiated Pauli-rotations: $U_{\text{test}} = \prod_k e^{-i\theta_k P_k}$. For the naive synthesis, U_{test} is the unitary of the reconstructed elementary circuit, i.e. the ordered product of the $\{R_Y, R_Z, \text{CNOT}\}$ gate operators embedded in the 2^n space.

Computing the reconstruction error is only feasible for small n . The operators act on a space of dimension 2^n , so U_{ref} and U_{test} each have 4^n entries, and the dense linear algebra required (matrix exponentials and matrix products) costs $\mathcal{O}(8^n)$. Already at $n = 7$ this is a 128×128 matrix ($4^7 = 16384$ entries); beyond this the cost grows excessively so we restrict verification on $n \leq 7$.

2. Gate count

The gate count measures how many elementary gates are needed to implement the unitary. In order to compare

the three models, we count the gates from a common elementary set, consisting of arbitrary single-qubit gates together with the two-qubit CNOT.

For the naive synthesis, this is trivial, as its output already consists of elementary gates $\{R_Y, R_Z, \text{CNOT}\}$, and we can simply count the gates produced.

The structure-preserving decomposition and Trotterisation instead output a list of Pauli rotations $e^{-i\theta_k P_k}$, where each P_k is a Pauli word. To compare them we count the elementary gates that the standard Pauli-rotation circuit produces for each word, rather than the gates explicitly. A Pauli rotation $e^{-i\theta P}$ on a word P of weight w (its number of non-identity factors) is implemented in five stages [1]:

1. a single-qubit basis change rotating every X or Y factor into the Z eigenbasis: a Hadamard for X , since $HXH = Z$, and a single $\frac{\pi}{2}$ rotation about x for Y , since it maps Y to Z likewise.
2. a descending CNOT ladder of $w - 1$ gates accumulating the parity of the w active qubits onto a single target.
3. a single $R_Z(\theta)$ on that target.
4. an ascending CNOT ladder of $w - 1$ gates undoing stage 2.
5. the inverse basis change undoing stage 1.

Each X or Y factor thus requires one basis-change gate before and one after the ladder. Writing $|\{X, Y\}|$ for the number of factors equal to X or Y , the single-qubit count per word is

$$\text{single} = 1 + 2|\{X, Y\}|,$$

where the 1 is the central R_Z rotation. The two ladders contribute

$$\text{CNOT} = 2(w - 1),$$

so the per-word total is

$$\text{total} = \text{single} + \text{CNOT} = 1 + 2|\{X, Y\}| + 2(w - 1).$$

The elementary gate count of the full decomposition is the sum of total over all its Pauli words.

3. Decomposition time

Lastly, the decomposition time is measured for each of the three methods. Note that this metric will vary depending on the hardware the code is run on. We ran on a MacBook Pro M4 Pro with 24GB of RAM.

For the structure-preserving decomposition, timing starts at the recursive BDI/KAK decomposition and stops once the resulting operators have been mapped back to Pauli rotations. It therefore excludes generating the Pauli generators, computing the dynamical Lie

algebra, the mapping to Majorana operators, the construction of the $\mathfrak{so}(2n)$ matrix, and its exponentiation to $U(t)$, as these are not part of the decomposition itself.

For the naive synthesis, timing starts at the recursive AIII/Type-A (KAK) decomposition and stops once the elementary $\{R_Y, R_Z, \text{CNOT}\}$ gate list has been produced. It excludes building the Hamiltonian H and the target unitary $U(t) = e^{-iHt}$ that is fed to the decomposition.

For Trotterisation, only the construction of the first-order Trotter sequence is timed. This step is essentially analytic, so the measured time is correspondingly small.

IV. RESULTS

This section presents the numerical results of the three different compilation methods, naive, SPD and Trotter, decomposing the same TFIM time-evolution $U(t)$. We start by validating the full structure-preserving pipeline on a small system with $n = 2$ against the exact unitary in section IV A. In section IV B we then study the convergence of the first-order Trotterisation as the number of Trotter steps is increased. Finally, in section IV C we compare the three methods in terms of the metrics described in section III D: Gate-count, reconstruction error and decomposition time.

A. Small-system validation of SPD

We first validate the structure-preserving pipeline on the two-qubit TFIM and compare it to the exact Hilbert-space unitary.

We use the rotated Hamiltonian presented in eq. (9) with parameters $J = h = 1$ and evolution time $t = 0.5$. Thus, the Hamiltonian is given as

$$H_{\text{rot}} = -X_0X_1 - Z_0 - Z_1,$$

where the initial generators are $\{X_0X_1, Z_0, Z_1\}$. Moreover, computing the DLA of these generators gives the same elements as listed in eq. (10). This is consistent with the dimension of $\mathfrak{so}(2n)$ for $n = 2$.

$$\dim(\mathfrak{so}(4)) = 2(4 - 1) = 6.$$

Using the Majorana representation and isomorphism in eq. (13), the skew-Hermitian generator $-iH_{\text{rot}}$ is mapped to the following skew-symmetric matrix

$$\rho(-iH_{\text{rot}}) = \begin{pmatrix} 0 & -2 & 0 & 2 \\ 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & -2 \\ -2 & 0 & 2 & 0 \end{pmatrix},$$

This is consistent with the matrix in eq. (14) for $J = h = 1$. Exponentiating this matrix, we get

$$O(t) = \exp(t\rho(-iH_{\text{rot}})) \in SO(4).$$

This is the matrix that we apply the recursive BDI decomposition to.

For $n = 2$, the BDI recursion consists of a single split,

$$SO(4) \longrightarrow SO(2) \times SO(2).$$

The four K -blocks are thus 2×2 special orthogonal matrices, each corresponding to one plane rotation. The middle Cartan factor A contains two commuting rotation angles. Hence the five BDI factors produce 6 Pauli rotations after mapping back to the qubit representation.

The resulting Pauli rotations are presented in table II. Here each row represents a rotation gate on the form

$$RP(\alpha) = e^{-i\alpha P}.$$

TABLE II. Pauli rotations obtained from the structure-preserving decomposition for $n = 2, J = h = 1$ and $t = 0.5$. Each row represents a gate $RP(\alpha) = e^{-i\alpha P}$. The column labelled “Factor” indicates whether the Pauli rotation originates from the left block-diagonal factor K_1 , the Cartan factor A , or the right block-diagonal factor K_2 in the BDI decomposition $G = K_1AK_2$.

Index	Factor	Pauli word	α
1	K_1	Z_0	0.5172
2	K_1	Z_1	1.3026
3	A	X_0Y_1	0.0861
4	A	Y_0X_1	-0.9139
5	K_2	Z_0	-1.0536
6	K_2	Z_1	1.3026

The corresponding circuit representation is shown in fig. 3. In the circuit, the two-qubit rotations come from the A Cartan block, while the K -blocks correspond to single-qubit rotations.

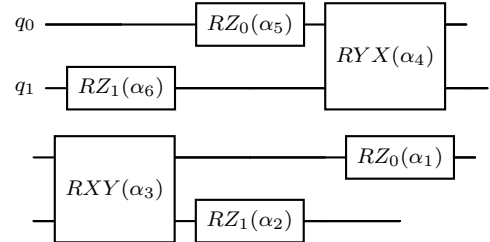


FIG. 3. Structure-preserving circuit for the two-qubit TFIM in the order applied by the circuit implementation. Here $RZ_j(\alpha)$ denotes $e^{-i\alpha Z_j}$, $RXY(\alpha)$ denotes $e^{-i\alpha X_0Y_1}$, and $RYX(\alpha)$ denotes $e^{-i\alpha Y_0X_1}$.

Finally, to verify the decomposition, we compare the decomposed unitary from the structure-preserving method to the exact time evolution operator

$$U_{\text{ref}}(t) = e^{-iH_{\text{rot}}t}.$$

Since the unitaries are physically equivalent up to a global phase, we use the phase-aligned reconstruction error in eq. (16). For $n = 2$, we obtain the error ϵ

$$\epsilon = 1.03 \times 10^{-15}.$$

This confirms that the SPD reproduces the exact time-evolution unitary up to numerical precision.

B. Trotter convergence with r

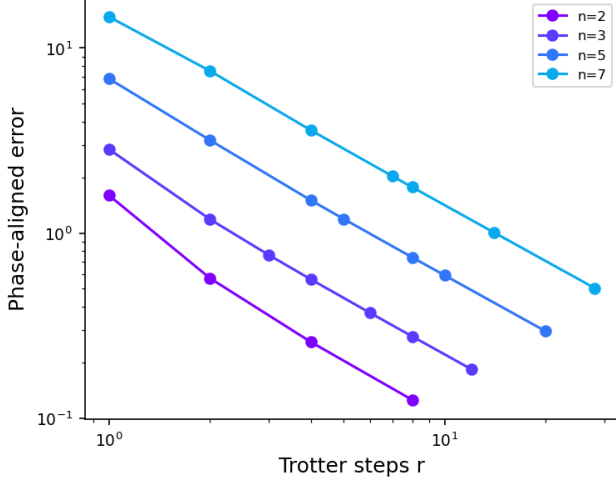


FIG. 4. Phase-aligned error as a function of Trotter steps r for four different qubit counts. The axes are logarithmic.

Figure 4 shows the phase-aligned reconstruction error as a function of the number of Trotter steps r for four qubit counts. Increasing r refines the first-order Trotter formula and thus reduces the error. We characterise Trotter's r -dependence here because the comparisons below fix $r = n$. On the log-log axes, the data lies on a straight line of slope ≈ -1 , which signifies a power-law decay $\text{err} \propto r^{-1}$, as expected for a first-order Trotter formula.

The error is consistently higher for larger qubit counts. This is because the Trotter error is governed by the commutators $[H_i, H_j]$ of non-commuting terms, and the number of such nearest-neighbour coupling pairs grows with the system size. Furthermore, as discussed in section III D 1, the Frobenius norm is not normalised by dimension, so the same per-amplitude error in the unitary is registered as a larger error as the Hilbert space grows. Empirically, the product $\text{err} \cdot r$ - the constant C in $\text{err} = C/r$ - rises from $C \approx 2.2$ at $n = 3$ to $C \approx 14$ at $n = 7$, which is consistent with the expected growth of the commutator norms and the unnormalised Frobenius norm.

Figure 5 extends the $n = 3$ curve to $r = 10^6$. The $1/r$ power law persists across the entire range. The error at the largest step count, $\text{err} = 2.20 \times 10^{-6}$ at $r = 10^6$, agrees with the small- r fit $\text{err} \approx 2.2/r$. Because the prefactor C grows with n , a fixed target accuracy needs proportionally more steps for larger systems, so taking $\text{err} \sim 10^{-6}$ at $r = 10^6$ as a practical best case for $n = 3$ is therefore an optimistic bound for larger n .

Figure 6 shows elementary gate count as a function

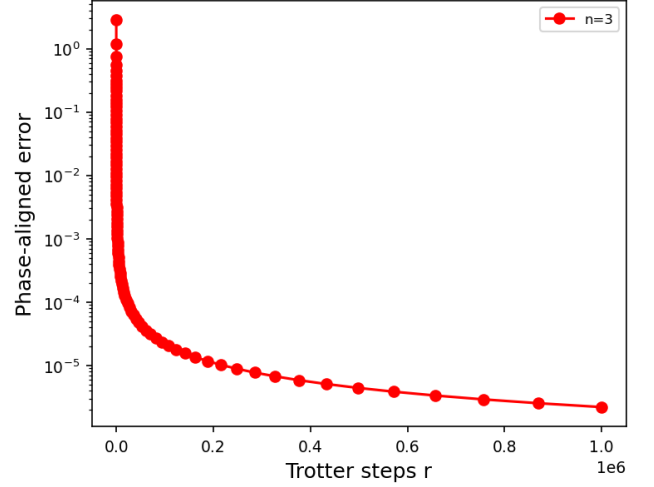


FIG. 5. Phase-aligned error as a function of Trotter steps r for $n = 3$ with $r \leq 10^6$. The y-axis is logarithmic.

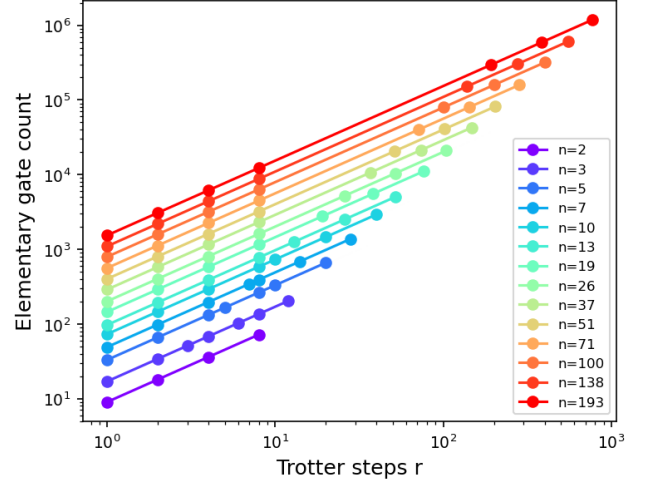


FIG. 6. Elementary gate count as a function of Trotter steps r for 14 different values $n \in [2, 193]$. The axes are logarithmic.

of the number of Trotterisation steps r . This is again a straight line on log-log plot, but now with a slope $+1$. The count is thus linear in r . This follows directly from the construction, where each Trotter step implements the $2n - 1$ Hamiltonian terms, contributing a fixed number of elementary gates per step. The total is therefore $(\text{gates per step}) \times r$. The per-step cost itself grows with n , which raises the line for larger n . The largest circuits reach $\sim 10^6$ elementary gates for $\sim 10^3$ Trotter steps r (for $n = 193$).

Combining the two scalings clearly illustrates the tradeoff between accuracy and complexity in Trotterisation. Since $\text{err} \propto 1/r$ while the gate count $\propto r$, the cost to reach a target accuracy scales as $\text{gates} \propto 1/\text{err}$: each additional decimal digit of accuracy costs roughly

a factor of ten in circuit size. Reaching machine precision with Trotter is thus computationally infeasible. The choice of r is therefore a balance between accuracy and circuit complexity.

C. Comparisons

1. Gate-count

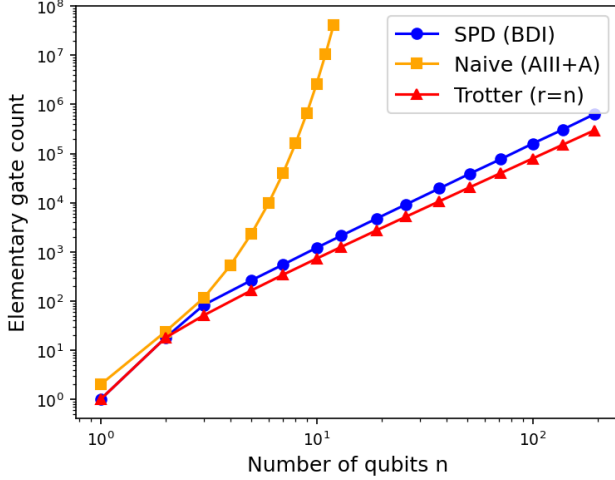


FIG. 7. Elementary gate counts as a function of the number of qubits n for the naive, SPD and Trotter methods. The axes are logarithmic.

In fig. 7 we compare the elementary gate count of the three different methods. The SPD and Trotter methods both scale polynomially in n , as seen by the straight line in the log-log plot. Trotter sits consistently a little below SPD. At the chosen depth $r = n$ the two methods emit the same number of Pauli rotations, $n(2n - 1)$, so the difference is due entirely to the *weight* of those rotations - the number of non-identity Paulis each acts on - which is lower for Trotter and determines how many elementary gates each rotation compiles to. For $n = 1$ and $n = 2$, Trotter and SPD are exactly the same, as the Pauli words have the same weight w .

The naive method, by contrast, scales exponentially, $\mathcal{O}(4^n)$, as expected for the synthesis of an arbitrary $2^n \times 2^n$ unitary. On the log-log axes this appears as a curve bending steadily upward, away from the straight polynomial lines of the other two methods.

Figure 8 shows the CNOT count. The qualitative picture is the same, with SPD and Trotter scaling polynomially, and the naive method scaling exponentially. However, the gap between SPD and Trotter is now larger, and widens with n (from $\sim 2\times$ at $n = 7$ to $\sim 4\times$ at $n = 193$, against only $\sim 2\times$ in the elementary gate count). Here, we omit $n = 1$, yet as in the elementary gate count they coincide for $n \leq 2$. The CNOT cost of a Pauli rotation is

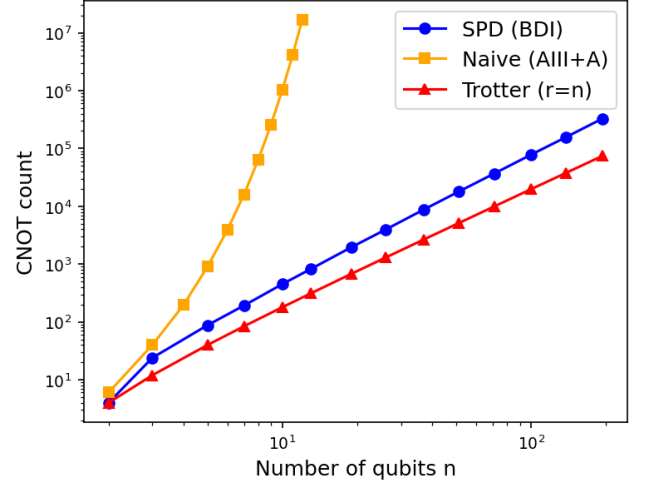


FIG. 8. CNOT count as a function of the number of qubits n for the naive, SPD and Trotter methods. The axes are logarithmic.

fixed by its weight w through the ladder length $2(w - 1)$. Trotterisation preserves the nearest-neighbour locality of the Hamiltonian, where every term is weight ≤ 2 and contributes at most two CNOTs. The structure-preserving decomposition instead produces Jordan-Wigner strings of the form $X_i Z_{i+1} \cdots Z_{j-1} X_j$, with X/Y only at the two endpoints but a Z -string in between, so their weight, and hence CNOT cost, grows with the number of qubits they span. The number of X/Y factors stays small, so the single-qubit counts of the two methods remain close ($\sim 1.4\times$). This is why the elementary-gate gap ($\sim 2\times$) is narrower than the CNOT gap ($\sim 4\times$).

Comparing the three methods, one should consider the reconstruction error as well as the gate count. As described in section IV B, the Trotter curves are taken at the fixed depth $r = n$, where the reconstruction error is of order unity (≈ 0.5 -2 over the qubit counts shown) (fig. 4), so it appears cheap only because, at this depth, it is a rough approximation. On the other hand, SPD and naive counts correspond to exact solutions.

2. Reconstruction errors

Figure 9 shows the phase-aligned reconstruction error for the three methods. For all three methods, the error increases as a function of the number of qubits. We omit the $n = 1$ case, as the Hamiltonian then has no coupling term and so all three methods are trivially exact. For the structure-preserving (SPD) and naive methods the error is $\sim 10^{-15} - 10^{-13}$, i.e. at the level of machine precision. Both are exact decompositions, so the error is purely numerical, accumulated by finite-precision linear algebra through the cosine-sine and Schur decompositions and the matrix exponential. Trotterisation, by contrast, is an

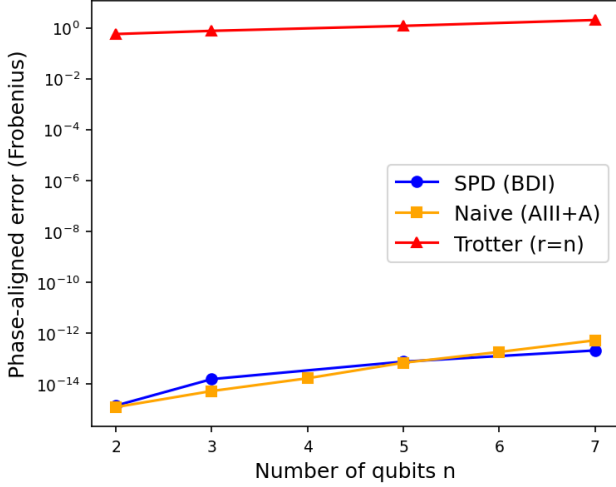


FIG. 9. Comparison of the phase-aligned reconstruction error as a function of the number of qubits n for the naive, SPD and Trotter methods. The y-axis is logarithmic.

approximate method, and at the depth plotted ($r = n$) its error is of order unity (≈ 0.5 -2 for $n = 2$ -7), roughly thirteen orders of magnitude larger. This is a genuine algorithmic error of the first-order product formula.

All three curves rise for increasing n , but for different reasons. The Trotter error rises because of the growth of the product-formula error with system size, (more non-commuting terms; section IV B). For the exact methods, the rising error comes from the accumulating error over the larger number of operations required at higher n , together with the accumulated Frobenius norm as discussed in section III D 1. In both cases, the exact methods stay at the precision floor.

3. Decomposition time

Figure 10 shows the decomposition times of the structure-preserving (SPD) and naive methods against the number of qubits, on log axes. In our implementation, the measured naive decomposition time scales exponentially as $\mathcal{O}(4^n)$ (fitted base ≈ 4), while the measured SPD decomposition time scales polynomially as $\sim n^{1.9}$ i.e. roughly $\mathcal{O}(n^2)$. This is the direct payoff of performing the decomposition in the smaller space $SO(2n)$, rather than in $U(2^n)$. The structure-preserving method operates on $2n \times 2n$ matrices whereas the naive method must manipulate the full $2^n \times 2^n$ unitary. The SPD curve has some fluctuations and is not perfectly smooth; at $n \lesssim 20$ the decomposition takes only a few milliseconds so the discrepant points are likely a result of measurement noise rather than any structural effect. Once the times exceed ~ 10 ms ($n \gtrsim 26$) the curve is smooth and follows the n^2 trend more closely.

We also compare the total pipeline time in fig. 11,

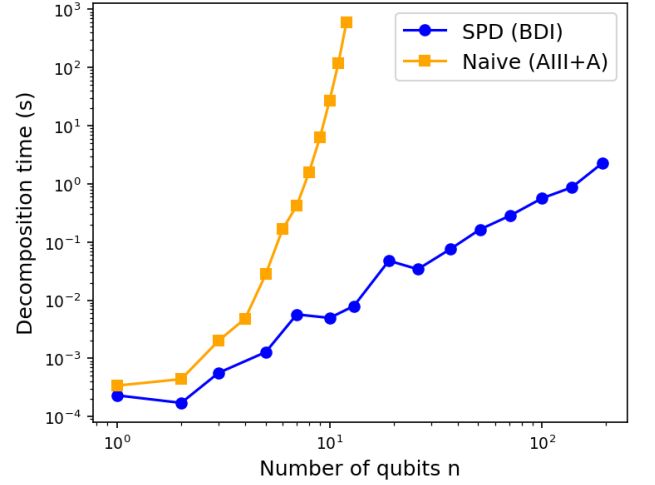


FIG. 10. Decomposition time as a function of the number of qubits n , for the structure-preserving (SPD) and the naive methods. The axes are logarithmic.

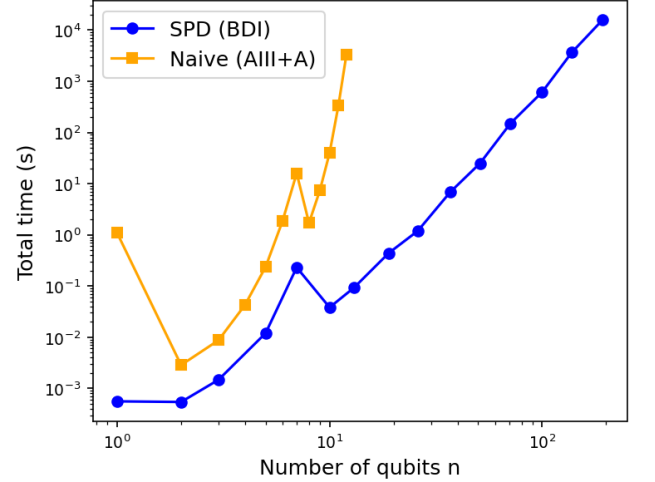


FIG. 11. Comparison of the total pipeline time as a function of the number of qubits n , for the naive and structure-preserving (SPD) methods. The axes are logarithmic.

which includes all setup and verification (for $n \leq 7$) rather than decomposition time alone. As only solutions with $n \leq 7$ are verified, we see that the earliest data points are slightly offset compared to the larger qubit counts. The naive method remains exponential, and is steeper than its decomposition time (approximating $\mathcal{O}(8^n)$) because it must additionally build and exponentiate the full $2^n \times 2^n$ Hamiltonian - matrix exponentiation costing $\mathcal{O}((2^n)^3) = \mathcal{O}(8^n)$. The SPD total time also grows faster than its decomposition time, scaling as $n^{3.4}$, compared with $n^{1.9}$ for the decomposition itself. At $n = 193$, the total pipeline takes several hours, while the decomposition itself takes only a couple of seconds.

The main bottleneck is the construction of the dynamical Lie algebra (DLA), which profiling shows to account for $\gtrsim 99\%$ of the total runtime. The DLA is built by repeatedly commuting the generating Pauli words and adding any new linearly independent words until the algebra closes (at dimension $n(2n - 1)$). The number of commutators to evaluate grows with the size of the expanding set, so this closure, rather than the decomposition itself, dominates the cost at large n .

D. Discussion of tradeoffs

Table III summarises the three methods across the metrics studied above.

TABLE III. Summary of the three decomposition methods. Gate counts are elementary $\{R_z, R_y, \text{CNOT}\}$; errors are the phase-aligned Frobenius norm. Trotter entries are at the fixed depth $r = n$ and are tunable in r . Largest n denotes the largest tested system size included in the numerical experiments.

Method	Gate count	Decomp. time	Error	Largest n
Naive	$\mathcal{O}(4^n)$	$\mathcal{O}(4^n)$	$\sim 10^{-13}$ (exact)	12
SPD	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\sim 10^{-13}$ (exact)	193
Trotter ($r = n$)	$\mathcal{O}(n^2)$	negligible	$\propto 1/r$	193

Three regimes emerge from the results. The naive method is fully general, decomposing a $2^n \times 2^n$ unitary but paying the generic $\mathcal{O}(4^n)$ cost for both the gate count and the decomposition time. Therefore it is only feasible for small systems, up to $n = 12$ in our experiments. It serves as a baseline that quantifies the cost for ignoring the structure. The structure-preserving method (SPD) keeps the decomposition exact while reducing both the gate count and decomposition time to be polynomial in n , by working in $SO(2n)$ rather than $U(2^n)$. This is the payoff of exploiting the free-fermionic DLA structure of the TFIM, evaluated up to $n = 193$. Trotter is the cheapest to produce, as its decomposition is analytic, and it inherits the Hamiltonian’s nearest-neighbour locality, but it is approximate and thus has a much larger error than the two exact methods.

V. CONCLUSION

In this report we have decomposed the transverse-field Ising Hamiltonian using three different methods of unitary synthesis. All methods successfully decompose the unitary, but with different tradeoffs. The naive implementation is exact and scales as $\mathcal{O}(4^n)$ for both the gate count and the decomposition time. The largest system size n decompose with this method is $n = 12$. The structure-preserving method is also exact, but exploits the structure of the DLA by working in the $SO(2n)$ subspace, leading to gate count and decomposition time scaling polynomially as $\mathcal{O}(n^2)$. With this method we manage to decompose an $n = 193$ -qubit unitary. Looking at the total pipeline time, however, we see that the SPD scales closer to $n^{3.4}$, as the generation of the DLA takes $> 99\%$ of the pipeline time. The Trotter method at $r = n$ also scales as $\mathcal{O}(n^2)$ for the gate count, but achieves a much *lower* decomposition time, as the calculation is analytical. This comes at the cost of an increased error, scaling as $1/r$, because the method is approximate. We also reach $n = 193$ qubits with this method.

Exploiting the structure of the target unitary can therefore allow for a faster and less complex decomposition than a naive approach. This advantage does, however, rely on the TFIM being free-fermionic, so that its DLA stays polynomial in size and embeds in $\mathfrak{so}(2n)$; for a generic Hamiltonian the DLA grows exponentially and the structure-preserving advantage is lost. When the model admits such structure and an exact decomposition is required, the structure-preserving method is the preferred choice. If exact reconstruction is not required, or the Hamiltonian has no exploitable structure, the use of Trotterisation as an approximate algorithm is a better choice, as the decomposition time is significantly lower. The naive method, applicable to any unitary but limited to small systems, serves here as a baseline that quantifies the cost of ignoring the structure.

Finally, future work could focus on reducing the cost of the DLA construction, which dominates the structure-preserving pipeline, on higher-order Trotter formulas for a better accuracy–cost tradeoff, and on extending the structure-preserving approach to other structured Hamiltonians.

REFERENCES

-
- [1] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.
 - [2] Sebastian Rietsch, Abhishek Y. Dubey, Christian Ufrecht, Maniraman Periyasamy, Axel Plinge, Christopher Mutschler, and Daniel D. Scherer. Unitary synthesis of clifford+(t) circuits with reinforcement learning. 2024.

- [3] H. F. Trotter. On the product of semi-groups of operators. *Proceedings of the American Mathematical Society*, 10(4):545–551, 1959.
- [4] Seth Lloyd. Universal quantum simulators. *Science*, 273(5278):1073–1078, 1996.
- [5] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. Theory of trotter error with commutator scaling. *Physical Review X*, 11(1):011020, 2021.
- [6] Brian C. Hall. *Lie Groups, Lie Algebras, and Representations: An Elementary Introduction*. Springer, 2003.
- [7] David Layden. First-order trotter error from a second-order perspective. *Physical Review Letters*, 128(21):210501, 2022.
- [8] Jean Gallier. Basics of classical lie groups: The exponential map, lie groups, and lie algebras. In *Geometric Methods and Applications*, volume 38 of *Texts in Applied Mathematics*, pages 367–414. Springer, New York, NY, 2001.
- [9] David Wierichs, Maxwell West, Roy T. Forestano, M. Cerezo, and Nathan Killoran. Recursive cartan decompositions for unitary synthesis, 2025.
- [10] Jose A. de Carvalho, Carlos A. Batista, Tiago M. L. de Veras, Israel F. Araujo, and Adenilton J. da Silva. Quantum multiplexer simplification for state preparation. *ACM Transactions on Quantum Computing*, 6(4):1–12, 2025.
- [11] Isaac D. Smith, Hans J. Briegel, and Hendrik Poulsen Nautrup. Necessary and sufficient conditions for universal gates with pauli strings and beyond, 2026.
- [12] Roeland Wiersema, Efekan Kokcu, Alexander F. Kemper, and Bojko N. Bakalov. Classification of dynamical lie algebras of 2-local spin systems on linear, circular and fully connected topologies. *npj Quantum Information*, 10:110, 2024.
- [13] Adelina Barligea, Matthew L. Sims-Goh, and Jakob S. Kottmann. Enabling lie-algebraic classical simulation beyond free fermions, 2026.
- [14] J. J. Sakurai and Jim Napolitano. *Modern Quantum Mechanics*. Cambridge University Press, 3 edition, 2020.
- [15] Kangle Li and Hoi Chun Po. Higher-dimensional jordan-wigner transformation and auxiliary majorana fermions. *Physical Review B*, 106(11):115109, 2022.
- [16] Jens Eisert. Advanced quantum mechanics: Chapter 8, lattice models and strongly correlated systems. Lecture notes, Freie Universitat Berlin, 2019.
- [17] Kohei Fukai, Balazs Pozsgay, and Istvan Vona. Solving models with generalized free fermions i: Algebras and eigenstates, 2026.
- [18] Paul Nelson. Notes on lie groups. Lecture notes, ETH Zurich, 2020.